TECHJAM 2026 · PROBLEM STATEMENT 4

Shopping Copilot

Conversational search over a 50,000-product catalogue.
Symbolic state for what the user said,
vector state for what the user meant.

Document: Technical design, architecture, and evaluation plan
Dataset: Amazon Reviews 2023 — Clothing, Shoes & Jewelry, frozen at 50,000 items
Runtime: fully in-memory · no external vector database · ≤2 LLM calls per turn

1. Problem statement

Keyword search assumes the shopper already knows the words for what they want. Frequently they don't.

A conventional e-commerce search box matches strings. Type "*black running shoes*" and it works. Type "*something for a beach trip*" and it collapses, because no product in the catalogue is labelled that way. The gap sits between how people describe what they want and how catalogues are indexed.

Three failures compound this:

- **Mode blindness.** A shopper who knows their constraints needs precision; a shopper who is exploring needs breadth. Systems tuned for one are actively bad at the other — and applying a filter to an exploratory query deletes the right answer on turn one.
- **Monotonic memory.** Shoppers revise. A system that only accumulates constraints, never revising them, drifts further from the target with each turn.
- **Unpriced questions.** Clarification is valuable but not free. A system that asks whenever it is uncertain burns the user's patience; one that never asks guesses badly.

> **THE TASK**
>
> Build a headless agent that converses with a shopper across multiple turns and surfaces the item they eventually purchase — ranked as high as possible, in as few turns as possible, within a hard limit of ten turns per session.

1.1 What the brief requires

Pillar	Requirement	Our mechanism
I Architecture	Dual-track intent routing; multi-route retrieval (keyword, category, vector) feeding LLM semantic ranking; in-memory.	Slot-count router gates hard filtering; three parallel retrieval streams unioned into one pool; feature scorer then optional LLM rerank.
II Dialog	State tracker handling information accumulation and intent override (slot erasure and rewriting); proactive clarification on over-generality.	Slot dictionary with write / overwrite / delete, plus canonical intent reconstruction each turn; clarification gated on expected information gain.
III Self-evolution	Personalized context distillation; runtime workflow re-orchestration.	Per-session profile term list; telemetry-driven adjustment of question selection and thresholds, validated by ablation.
IV Evaluation	Hit Rate@10, MRR, MTTC, Efficiency.	Supplied evaluator, plus our own per-component ablation harness and per-scenario slicing.

1.2 Constraints we designed against

Constraint	Design consequence
Max 10 turns; exceeding scores zero	Hard cap on clarifications; every question must justify its cost.
Catalogue strictly read-only	All indexes derived and held separately; no mutation, no synthetic ASINs.
No external vector DB clusters	Brute-force exact kNN over an in-memory matrix. At 50k rows this is also the correct engineering choice.
No base-model fine-tuning	Pretrained sentence encoder used frozen; only a nine-weight scorer is fitted.
Text only	Product images ignored; all signal drawn from title, features, description, categories.
No hosted model access provided	Pipeline is fully functional with zero LLM calls; LLM components sit behind flags.

2. What the data actually says

Four measurements on the supplied files changed the design substantially. Each is reproducible from `catalog.jsonl` and `public_set.jsonl`.

2.1 Purchase targets are overwhelmingly popular

Measurement	Value
Median `rating_number` of the 200 ground-truth targets	**6,846**
Median `rating_number` across the full catalogue	**12**
Targets in the top 1% of the catalogue by review count	**63%**
Targets in the top 5% / top 20%	81% / 96%

> **WHY THIS HOLDS ON THE PRIVATE SET**
> Sessions are anchored on real purchases, and real purchases concentrate on popular products. This is a property of consumer behaviour, not of the public split, so we expect it to transfer. It is nevertheless treated as a *soft prior* throughout — popularity boosts scores and contributes candidates, but never filters, because 4% of targets sit outside the top 20%.

2.2 Price is unusable as a filter

78.9% of catalogue rows have `price: null`. A price filter would delete four-fifths of the catalogue, and with it the target in most sessions. Price is therefore a scoring feature with a neutral value for nulls, and never a constraint.

2.3 Structured attributes barely exist

Field	Coverage	Usable as
`categories`	100%	hard filter, keyword, embedding
`store` (brand proxy)	99.4%	filter / score
`details.Department`	87%	hard filter
`features` / `description`	89.6% / 52.2%	keyword, embedding
`price`	**21.1%**	score only
`details.Color` / `Material`	4.9% / 4.1%	unusable
size	**absent**	—

A representative long-tail row states "100% Polyester" and "One Size Fits Most (1X to 3X)" in its description while carrying neither as a field. Attribute matching must therefore operate over **text**, not structured lookup — which makes slot matching a scoring signal rather than a filter.

2.4 The user profile is mostly non-discriminative

Preference tags initially appeared predictive: sessions tagged `comfort` had targets mentioning comfort 79.9% of the time versus 40.5% for a random product. However, targets are popular, and popular products carry longer descriptions (1,240 vs 761 characters), so they match any keyword more often by chance.

Re-testing against a **popularity-matched baseline** (the 500 most-reviewed catalogue items) isolates the tag's real contribution:

Tag	Sessions	P(kw | target)	P(kw | popular)	Lift
fit	163	57.1%	62.2%	**−5.1**
material	154	81.8%	82.2%	−0.4
comfort	144	79.9%	78.6%	+1.3
style	101	54.5%	52.8%	+1.7
durability	47	27.7%	33.8%	**−6.1**
performance	26	50.0%	38.4%	**+11.6**
warmth	18	27.8%	16.2%	**+11.6**
weather	12	25.0%	15.4%	**+9.6**

The four tags carried by most profiles sit within ±2 of chance. Only three rare tags show real lift. We also verified that these

do *not* correspond to coherent categories — `warmth` targets span tops, underwear, socks and novelty, with the modal category covering just 2 of 18 — so the mechanism is **term presence in product text**, not a category prior.

Additionally, `purchase_frequency` is the constant string `"3-4 prior purchases"` across all 200 sessions, and `category_bucket` is constant `"clothing"`. Both carry zero information.

2.4.1 Rating style does carry signal

Separately from the tags, `rating_style` proves genuinely predictive. Users whose prior ratings skew positive purchase higher-rated products than critical users do:

rating_style	Sessions	Target mean [illegible]	Note
usually positive	134	**4.413**	Difference 0.131 stars. Permutation test over 20,000 shuffles: **p = 0.003**.
mixed	21	4.305	
critical	45	4.282	

However, `rating_style` and `average_prior_rating` are the same field. The mapping is deterministic: `5.0 → usually positive`, `4.0 → mixed`, `≤3.0 → critical`. Only one may be used; including both would introduce perfect collinearity on a 200-session fit.

> **EFFECT SIZE, HONESTLY STATED**
>
> The result is statistically real but modest: 0.131 stars against a catalogue interquartile range of 3.80–4.60, roughly 16% of one IQR. It is retained as an *interaction* feature (`rating × profile_expects_high`) rather than a standalone signal, since `average_rating` is already feature four. The regression is permitted to zero its weight, and it is ablated like everything else. We include it because it is measurably present in **100% of sessions**, unlike `rare_tag_match` at 22%.

> **A CORRECTION WE MADE UNDER MEASUREMENT**
>
> This feature was initially cut on the reasoning that rating style describes *how* a user rates rather than *what* they buy. That reasoning was wrong, and testing it took ten minutes. It is recorded here because the same discipline that removed four preference tags also restored this one — the method is measurement, not a bias toward simplicity.

> **CONSEQUENCE**
>
> The personalization subsystem we originally designed — profile embedding, [illegible] feature, per-dimension suppression mask, decay term — was reduced to **two ranking features**: a three-tag term list active in ~22% of sessions, and a rating-style interaction active in 100% (§2.4.1). The measurement, not the schedule, drove both the cut and the one restoration.

2.5 Sessions contain no dialogue

Each session in `public_set.jsonl` carries only `sample_id`, `scenario_type`, `difficulty_bucket`, `category_bucket`, `ground_truth.parent_asin`, and `user_profile`. User turns are generated at runtime by a simulator holding a hidden intent card.

Two consequences: training data for the ranker must be produced by an **instrumented run** rather than read from disk; and `scenario_type` is a scoring label, not an inference-time input.

Note also that `scenario_type` and `difficulty_bucket` are perfectly collinear on the public set (buying→easy, browsing→medium, intent_override→hard, boundary→medium), so they carry identical information. We do not assume this holds privately.

3. Architecture

Two representations of the conversation, three retrieval streams, one ranking stage.

3.1 Design thesis

Most conversational retrieval designs pick a single representation. Each fails in a specific, predictable way.

Approach	Strength	Failure
Pure symbolic slot filling, hard filters	Exact constraints; clean revision; fully explainable.	Cannot represent "*something for a beach trip*" — no product carries the concept as a field, so there is nothing to filter on.
Pure vector embed turns, accumulate, kNN	Captures meaning; graceful on vague scenario queries.	**Cannot forget.** A weighted sum of turn embeddings is monotonic: "*actually not black, blue*" leaves black permanently present in the query vector.

The second failure is disqualifying, because intent override is named in the brief and constitutes 15% of sessions. We therefore run both layers and wire the same event into each.

CORE DESIGN

A slot dictionary holds what the shopper literally said. It supports write, overwrite and delete, and gates hard filtering.

A canonical intent string is re-rendered from that dictionary every turn and embedded fresh. Because the dictionary no longer contains the superseded value, the rebuilt string cannot contain it either.

Three retrieval streams — keyword, semantic, popularity — contribute candidates independently and are unioned. Their blind spots do not overlap, so a miss requires all three to fail simultaneously.

DEVIATION FROM THE OBVIOUS DESIGN

An earlier version of this architecture used a **gated fusion** of the current utterance with a recency-decayed history vector, with the gate opening on override detection. We removed it. The symbolic layer already knows which constraints are live, so reconstructing the intent string is deterministic, inspectable (the string can be printed and read), and eliminates three tuned parameters we cannot fit on 200 sessions. Suppressing stale intent mathematically solves a problem that state reconstruction avoids having.

3.2 Offline stage

Runs once at startup, in roughly 5 seconds plus encoding time. Produces three views of one catalogue, all derived from a **single shared `product_text()` function** so that the indexes cannot silently diverge.

```
product_text(r) = title + features + description + categories + store + details[Department, Material, Color, Brand, Style]
```

Index 1 — Keyword (BM25)

Technology: SQLite FTS5, `unicode61` tokenizer with diacritic removal. Supplied in the participant kit; we extend rather than replace it.

Column weighting is already tuned in the baseline: title 6.0, categories 4.0, features 2.5, details 2.5, store 1.5, description 1.0. Note that SQLite's `bm25()` returns *negative* scores, more negative being better; we normalise sign once at the retrieval boundary so all downstream code treats higher as better.

WHY KEEP KEYWORD SEARCH AT ALL

BM25's IDF term explicitly rewards rarity. In our catalogue `everboots` appears in 1 product (idf 10.41) and `comfortable` in 10,562 (idf 1.55). Embeddings do the opposite — compressing ~1,200 characters into 384 dimensions discards precisely these rare, discriminative tokens. On the query "*Saucony Cohesion 10*", BM25 returns Cohesion 10, then 9, then 6, correctly ordered. A semantic encoder returns "some running shoe." Brand names, model numbers and exact specifications are where keyword retrieval is irreplaceable.

Index 2 — Semantic (dense vectors)

Technology: `sentence-transformers/all-MiniLM-L6-v2`, 384 dimensions, used frozen. Chosen for size (~80MB), CPU inference speed, and strong retrieval performance at this scale; fine-tuning is out of scope per the brief.

All 50,000 product texts are encoded once into a `(50000, 384)` float32 matrix, L2-normalised so cosine similarity reduces to a dot product. Resident size ~75MB. The matrix is persisted as `.npy` so subsequent runs load in under a second.

Search is brute-force exact kNN: `scores = catalog_matrix @ q`, then `np.argpartition` for top-k. Approximately 5 ms.

> **WHY NOT FAISS, HNSW OR A VECTOR DATABASE**
>
> Approximate nearest-neighbour indexes earn their cost at millions of vectors. At 50,000 rows a single BLAS call over 75MB returns *exact* results in milliseconds. An ANN index would add a dependency, a build step, tuning parameters, and an approximation, in exchange for saving roughly four milliseconds. The brief additionally prohibits external vector DB clusters, so brute force is both the compliant and the correct choice.

Index 3 — Structured facts

Technology: plain Python dictionaries. One record per ASIN holding `dept`, `cat3`, `store`, `price`, `rating_number`, precomputed `pop = log1p(rn)/log1p(100000)`, `rating`, and a lowercased text blob for substring matching.

A companion structure groups ASINs by category path with each list **pre-sorted by review count**, making "bestsellers in this category" a list slice rather than a scan.

Department is taken from `categories[1]` (100% coverage) rather than `details.Department` (87%).

3.3 Session state

Object	Contents	Update rule
Slot dictionary	Attributes the user explicitly stated.	Write on new key; **overwrite** on conflict, setting an override flag; **delete** on detected negation.
Scenario buffer	Un-slotted intent ("*for a beach trip*").	A new scenario statement **replaces** the previous one rather than appending.
Profile terms	Term list derived from rare preference tags.	**Read-only.** Loaded once in `reset()`; no code path writes to it.
Telemetry log	Per-turn pool sizes, question payoff, feature values.	Append-only.

> **INVARIANT: INFERRED PREFERENCE NEVER ENTERS THE SLOT DICTIONARY**
>
> The slot dictionary's value lies in containing *exactly* what the user said. Contaminating it with inferred preference would silently break intent override — our principal differentiator — and the failure would be invisible in aggregate metrics. The profile is held in a separate read-only object.

3.4 The turn loop

1 State update

Extract attributes from the incoming utterance and merge into the slot dictionary via write / overwrite / delete. Un-slotted content replaces the scenario buffer.

> **TECHNOLOGY**
>
> Primary path: one constrained-JSON LLM call returning a flat attribute object. Fallback path: regex plus a gazetteer built from the catalogue's own attribute vocabulary, which guarantees every extracted value exists in the data. Negation detection is pattern-based (`not X`, `instead of X`, `actually Y`). The fallback path alone yields a fully functional agent, so the system degrades gracefully without model access.

2 Canonical intent reconstruction

Re-render the full request from current state and embed it fresh.

> **TECHNOLOGY**
>
> `canonical = render(slots) + scenario_buffer`, encoded by the same MiniLM model used for the catalogue, so query and products occupy one space. One encode per turn, ~5 ms on CPU. Deterministic and printable — the exact string is written to the telemetry log, which makes override behaviour directly auditable.

3 Intent routing

Classify the turn as **buying** or **browsing** from slot-dictionary state.

> **TECHNOLOGY**
>
> Rule over slot state: presence of a department slot, or two or more hard constraints, or a leaf-category noun → BUYING; otherwise BROWSING. Re-evaluated every turn so the track can flip mid-session. No learned classifier: there are no routing labels available at inference time, routing is not the accuracy bottleneck, and a rule is trivially inspectable.

> **WHAT THE TRACK DECISION ACTUALLY CONTROLS**
>
> Filtering is the only irreversible operation in the pipeline. The router is therefore best understood as a **permission gate on filtering**: does the user's stated evidence justify permanently deleting candidates? Stream quotas also shift (buying favours keyword, browsing favours semantic), but that is a soft tilt; the filter decision is the substantive one. We scoped routing to the decision that is genuinely irreversible rather than manufacturing differences the catalogue's sparse attributes cannot support.

4 Multi-stream retrieval

Three independent streams, unioned and deduplicated into one candidate pool.

Stream	Buying quota	Browsing quota	Filters?
Keyword (FTS5)	120	60	department + category, buying only
Semantic (kNN)	40	150	never
Popularity	20	20	never

TECHNOLOGY

Keyword: FTS5 `MATCH` with over-fetch (3× limit) before filtering, so post-filter truncation cannot leave an undersized pool. Semantic: matrix multiply plus `argpartition`; MMR or per-category caps applied on the browsing track for diversity. Popularity: read the modal categories of the existing pool, slice the pre-sorted per-category lists. **Floor check** after all filtering — if the pool is undersized, relax the least-confident constraint and re-retrieve.

WHY UNION, AND WHY BROWSING NEVER FILTERS

Hit Rate@10 is bounded above by pool recall; nothing downstream recovers an item filtered out on turn one. Union is monotonic — it can only add candidates — so an extra stream carries no recall risk, only a marginally larger pool for a ranker that handles 200 comfortably. Conversely, on an open-ended query every constraint is inferred rather than stated, so the expected cost of a wrong filter (permanent loss of the target) exceeds any benefit. The three streams also fail differently: keyword is blind to paraphrase, semantic to rare specifics, popularity to the query entirely. A missed target requires all three to fail at once.

5 Clarification decision

Return ten recommendations every turn; additionally set `ask_attribute` when a question would earn its cost.

TECHNOLOGY

For each unfilled attribute, compute Shannon entropy over its value distribution across the candidate pool, then weight by an answerability prior:

```
score(a) = P(answerable | a) × H(a) H(a) = −Σ_v p(v) log₂ p(v)
```

Ask about `argmax` if the score clears a threshold. Answerability is initially hand-set and subsequently estimated from instrumented simulator runs. A per-session cap on clarifications protects MTTC, and asked attributes are recorded to prevent repetition.

WHY ENTROPY ALONE IS THE WRONG CRITERION

Measured on a real 200-candidate pool for "*warm winter jacket*": brand has the *highest* entropy at H=7.26 across ~180 distinct values, while category sits at H=3.34. Naive entropy would always ask about brand. Two terms defeat it. First, most shoppers hold no brand preference, so the answer is "no preference" and a turn is spent for zero reduction. Second, only three options can be surfaced from 180, so the true value is usually absent — and a user selecting the nearest wrong option applies a filter that *deletes the target*. A bad question costs Hit Rate, not merely MTTC. Weighting by answerability inverts the ranking correctly: category 2.84, department 1.67, brand 0.73.

INTERFACE FINDING: ASKING IS NEARLY FREE

`ask_attribute` and `recommendations` occupy the same return object, so a clarifying turn still returns a full ranking and is still scored. The decision is therefore not "ask *or* answer" but "answer, and additionally ask when useful" — which permits a markedly more permissive threshold than a mutually-exclusive design would.

6 Ranking

Score all candidates on ten features, truncate to 30, optionally rerank with an LLM, return 10.

Feature	Definition	Rationale
`bm25_norm`	FTS5 score, sign-corrected and max-normalised	lexical relevance
`cos_sim`	dot product with the query vector	semantic relevance
`pop`	`log1p(rating_number)/log1p(1e5)`	purchase-frequency prior (§2.1)
`rating`	`average_rating / 5`	quality signal
`price_fit`	0.5 when null, else fit to stated budget	null-safe; never excludes (§2.2)
`category_match`	matches slot category	constraint satisfaction
`brand_match`	`store` matches slot brand	constraint satisfaction
`slot_coverage`	fraction of slot terms present in the text blob	text-based attribute matching (§2.3)
`rare_tag_match`	1 if any rare-tag term present, else 0	surviving profile signal, ~22% of sessions (§2.4)
`rating_style_fit`	`average_rating` × profile rating skew	measured interaction, 100% of sessions (§2.4.1)

Ten features. `average_prior_rating` is deliberately excluded as collinear with `rating_style`.

TECHNOLOGY

Logistic regression (scikit-learn, `class_weight='balanced'`, L2 regularisation) over standardised features. Eleven parameters including the intercept, fitted on roughly 4,200 rows. Optional final pass: one LLM call carrying the dialogue, slot dictionary and 30 candidate titles, returning a reordering; parsed defensively with fallback to the regression ordering.

WHY TEN FEATURES AND NOT A LEARNED RANKER

Supervision consists of 200 sessions with a single positive each, no click stream, and a private evaluation set drawn from *different users and different products*. Learned attention heads or a trained gating network would fit this and fail invisibly on transfer. Eleven parameters against ~4,200 rows is roughly 380 examples per parameter. Note also that the rows are not independent — they derive from 200 sessions — so the effective sample size is nearer 200; we regularise accordingly and split by session under `GroupKFold`.

WHY THE LLM OPERATES ON 30 CANDIDATES, LAST

It is the only component that reads the conversation as language. Constraints such as "*nothing too flashy*" or "*he's always cold*" produce no slots and no features, yet are real. Restricting it to a shortlist bounds both the token cost (reported in `usage` and scored under Efficiency) and the damage from a misread — a deterministic scorer retains control of the coarse ordering. The component sits behind a flag and is retained only if ablation shows it earns its token cost.

7 Telemetry

Log per-turn state, pool sizes, question payoff, and per-candidate feature values.

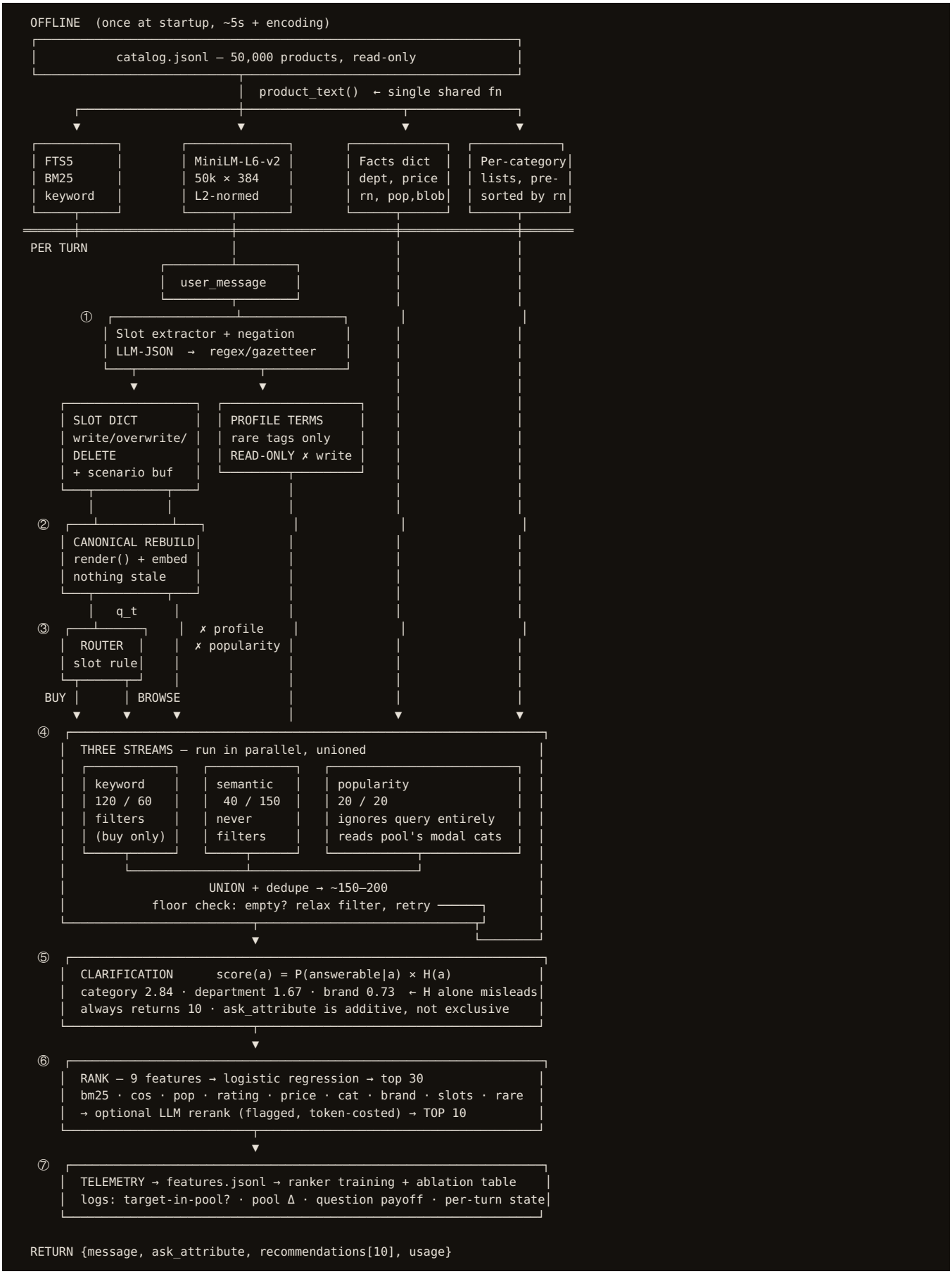
TECHNOLOGY

Append-only JSONL. Records `session_id`, `turn`, `track`, canonical intent string, pool size before and after filtering, `ask_attribute`, **whether the target was present in the pool**, and the ten feature values per candidate. Ground-truth labels are joined offline by `session_id`, never passed into `respond()`.

TELEMETRY IS A PREREQUISITE, NOT AN AFTERTHOUGHT

Because sessions ship without dialogue, this log is the *only* source of ranker training data — an instrumented run produces the feature matrix as a side effect. It also drives runtime adaptation (Pillar III) and supplies the ablation table (§5). We build the harness before the features it measures.

4. System diagram



5. Why this system is better

5.1 Against a keyword baseline

The supplied starter agent conflates retrieval and ranking — it issues one FTS5 query with `LIMIT top_k` and returns that order directly. On the query "*warm winter fleece jacket men*" its top five carry review counts of 2, 3, 156, 80 and 1. Given that 63% of targets sit in the top 1% of the catalogue by review count, those are almost certainly wrong, and keyword relevance alone cannot detect it.

Separating retrieval from ranking and adding the popularity feature reorders to 156, 539, 307, 3. The item with the *best* keyword score falls to fourth — but remains on the list, because popularity boosts rather than prunes.

5.2 Against a pure-vector design

Accumulating turn embeddings cannot represent deletion. After "*black shoes*" then "*actually not black, blue*", decay reduces black's weight but never removes it, and the stale intent drags forward for the remainder of the session. Canonical reconstruction rebuilds the query from a dictionary that no longer contains black, so there is nothing to suppress. This is the behaviour named in Pillar II and it constitutes 15% of sessions, all labelled `hard`.

5.3 Against a more elaborate design

We removed more than we added. Each removal was driven by measurement:

Removed	Reason
Gated fusion of history	State reconstruction achieves the same outcome deterministically, with three fewer tuned parameters and a printable intermediate.
Profile embedding, `profile_match`, suppression mask	Four of five common tags measured at or below chance against a popularity-matched baseline (§2.4).
Price filtering	78.9% of the catalogue has no price; filtering would delete the target in most sessions.
Structured attribute filtering	`Color` 4.9%, `Material` 4.1%, size absent. Attribute matching moved to text scoring.
Multi-head attention, learned gating	Unfittable on 200 sessions; private set uses different users and products.
Rare-tag category prior	Hypothesised, then checked: `warmth` targets span tops, underwear, socks; modal category covers 2 of 18. Reduced to term matching.

THE GOVERNING PRINCIPLE

Irreversible operations require explicit evidence; weak evidence gets reversible operations only.

Filtering deletes permanently, so it acts only on stated constraints over well-populated fields. Ranking reorders a pool that still contains the target, so weak signals — popularity, profile terms, LLM judgement — are welcome there. Nearly every decision in this document is that rule applied at a different point.

6. Metrics and how we test them

6.1 Metric definitions

Hit Rate@10 — Coverage $\mathrm{HitRate@10} = (1/N)\ \Sigma_i\ 1[\ \mathrm{target}_i \in \mathrm{top10}_i\ ]$

Fraction of sessions where the purchased item appears anywhere in the returned ten. Measures retrieval recall and boundary capability.

Governed by: Step 4. Bounded above by pool recall — if the target is absent from the pool, no downstream stage recovers it.

MRR — Precision $\mathrm{MRR} = (1/N)\ \Sigma_i\ 1\ /\ \mathrm{rank}_i$

Mean reciprocal of the target's position. Rank 1 scores 1.0, rank 5 scores 0.2, absent scores 0.

Governed by: Step 6. Conditional on the target being in the pool.

MTTC — Efficiency of conversation $\text{MTTC} = (1/N)\ \Sigma_i\ \text{turns\_to\_conversion}_i$

Mean turns to reach the correct product. Lower is better. Sessions exceeding 10 turns terminate with zero score.

Governed by: Step 5. Because `ask_attribute` is additive rather than exclusive, a question costs a turn only when the session would otherwise have converged sooner.

Efficiency — Computational cost

Derived from the `usage` object returned each turn. Our pipeline reports zero tokens with LLM components disabled; the extractor and reranker are the only consumers.

Governed by: Steps 1 and 6, both flag-controlled.

6.2 Internal diagnostics

Two measurements not scored by the evaluator, but decisive for debugging:

Diagnostic	Why it matters
Pool recall was the target in the pool at all?	The single most informative signal. If pool recall is 0.55 and Hit@10 is 0.30, ranking is the bottleneck. If pool recall is 0.35, no amount of ranking work helps and Step 4 needs attention. This boolean tells you which half of the pipeline is broken.
Pool shrinkage per question	Validates the clarification policy. A question that does not reduce the pool has cost a turn for nothing, and its attribute should be deprioritised.

6.3 Ablation protocol

Each configuration is run over all 200 public sessions and scored with the supplied evaluator. We ablate *decisions*, not every feature — the regression weights already reveal a feature's contribution, and a near-zero weight is itself evidence.

Configuration	Question answered	Read primarily	Expected direction
Full pipeline	baseline	all	—
− `rating_style_fit`	Does the measured rating-style effect survive in the pipeline?	**MRR**	Small; effect size is 16% of one IQR
− popularity feature	Is the 63%-in-top-1% finding operative?	**MRR**	MRR falls
− popularity stream	Does the fallback catch targets the text streams miss?	**Hit@10**	Hit@10 falls
− semantic stream	Does the embedding work justify itself?	**Hit@10**	Hit@10 falls, most on browsing
− keyword stream	Does BM25 still contribute given embeddings?	**Hit@10**	Hit@10 falls, most on buying
− department filter	Does filtering help or hurt on balance?	**Hit@10 *and* MRR**	Hit@10 may rise while MRR falls — the trade must be read jointly
− LLM rerank	Does it beat its token cost?	**MRR vs Efficiency**	MRR falls, Efficiency rises sharply
− `rare_tag_match`	Is the rare-tag signal worth anything?	**MRR**, on the 22% subset	Small or null

TWO PROTOCOL NOTES

Feature ablations need no re-run. Zeroing a weight and re-scoring the logged candidates takes seconds; only stream and filter ablations change pool composition and require a full pass.

The LLM row is the only genuine trade-off. Every other component either helps or does not. The reranker improves MRR while degrading Efficiency, so the decision depends on their relative weighting inside `TechnicalScore` — which we read from the evaluation config rather than assume.

6.4 Slicing

Every metric is additionally reported by `scenario_type`. Aggregate scores conceal which capability is failing:

Slice	n	What a failure here indicates

buying	80	Filtering or keyword weighting is wrong.
browsing	80	Semantic retrieval or diversity is wrong.
intent_override	30	Negation detection or state reconstruction is failing. All 30 are `hard`; this is the highest-leverage slice.
boundary	10	Undocumented edge behaviour. Defensive coding only — see §7.

6.5 Generating the training corpus

Sessions ship without dialogue, so the feature matrix cannot be read from disk. It must be produced by running sessions and logging what happens. Where the user turns come from depends on what the kit supplies — a question that must be resolved before any of this is planned.

6.5.1 Resolving the dialogue source

The local evaluator is described as deterministic and scores MTTC, which implies multi-turn interaction. Three configurations are possible, and the correct response differs materially between them. This is the first thing we establish.

If the kit provides…	How to tell	What we do
A. A bundled user simulator	Evaluator imports a simulator or user-policy module; intent cards ship separately from `public_set.jsonl`.	Run it directly. Log inside `respond()`. No additional work.
B. Single-turn evaluation only	Evaluator calls `respond()` once per session; MTTC is trivially 1 locally and only meaningful on the private harness.	Training data is straightforward but covers one turn. We build our own simulator (§6.5.2) to exercise multi-turn and override paths.
C. No simulator at all	Evaluator scores a submitted ranking; the participant supplies the conversation loop.	We build the simulator ourselves (§6.5.2). It becomes a required component rather than a convenience.

> **WHY THIS IS RESOLVED FIRST**
>
> Under configuration A the training corpus is a logging change. Under C it requires building a user simulator before any weight can be fitted, which moves work onto the critical path and changes the schedule. The distinction is settled by reading the evaluator source (step D2), which takes minutes and removes a dependency-shaped unknown from the plan.

6.5.2 Fallback: a self-built user simulator

If the kit supplies no simulator, one can be constructed from information we already hold. Each session gives the target ASIN, and the catalogue gives that product's full record — title, categories, department, store, features. A plausible shopper for that session is one progressively describing that product.

```
target = "London Fog Men's Auburn Zip-Front Golf Jacket" categories = [... , Men, Clothing, Jackets & Coats] turn 1 broad "I'm looking for a men's jacket" turn 2 narrow "something for cooler weather" turn 3 specific "zip front, not a hood"
```

Turn content is drawn from the target's own attributes, released progressively rather than all at once. Scenario type governs the release policy:

[illegible]	Simulator policy
buying	Emit two or more concrete attributes on turn 1; add detail steadily.
browsing	Open with a scenario phrase carrying no extractable attribute; concede specifics only when asked.
intent_override	State an attribute the target does *not* have, then contradict it on a later turn with the true value. This is the path that exercises slot deletion.
boundary	Withhold most attributes; answer clarifications minimally.

When `ask_attribute` is set, the simulator answers from the target's record if it carries that attribute, and replies "no preference" otherwise — which is what makes the answerability prior in Step 5 measurable.

> **WHAT A SELF-BUILT SIMULATOR CAN AND CANNOT ESTABLISH**
>
> It is **fit for purpose** as a source of feature vectors, because a training row needs a query, a pool and a label — and any plausible query produces all three. It is also adequate for exercising the override path and for regression-testing slot deletion.
>
> It is **not** a substitute for the real evaluator when reading scores. Our simulator's phrasing derives from the target's own title, which makes retrieval easier than reality and inflates Hit Rate. Absolute numbers from a self-simulated run are therefore reported as internal diagnostics only; all headline figures come from the supplied evaluator. Relative comparisons — ablations, A/B of weights — remain informative because the bias applies equally across configurations.

> **MITIGATING THE PHRASING BIAS**
> Where an LLM is available, turns are paraphrased rather than copied from the title, which restores some vocabulary mismatch. Where it is not, the simulator draws from `categories` and `details` rather than `title`, since category vocabulary is generic and less likely to be echoed verbatim in the product text.

6.6 Ranker training protocol

Because sessions ship without dialogue, training data must be generated:

1. Run the evaluator over all 200 public sessions with telemetry enabled.
2. Each turn logs one row per candidate: `session_id, turn, parent_asin, [10 features]`.
3. Join `ground_truth` offline by `session_id` to assign labels — 1 for the target, 0 otherwise.
4. Sample ~20 negatives per session **from the candidate pool**, not from the catalogue at large.
5. Fit logistic regression on the resulting ~4,200 rows; validate with `GroupKFold` grouped by `session_id`.
6. Re-run with fitted weights and confirm improvement against the hardcoded baseline.

> **WHY NEGATIVES ARE SAMPLED FROM THE POOL**
> Random catalogue negatives are trivially separable — an obscure 3-review product is obviously not the answer — so the model would learn only "popular is good." Pool negatives are the candidates that nearly won, which is the discrimination the ranker actually needs to perform at inference time.

> **OBJECTIVE FUNCTION**
> Logistic regression optimises binary cross-entropy, which scores each item independently, whereas MRR is a ranking objective over items within a session. The mismatch is real but modest at nine features. If time permits we compare against a pairwise objective (`lambdarank`, grouped by session) and keep whichever wins on MRR. A third option, given only nine weights, is direct coordinate ascent on MRR over the development set — crude, but it optimises the scored metric with no proxy.

7. Team structure and delivery plan

Five contributors, three build days. The split follows the module boundaries in §3 so that work proceeds in parallel against agreed interfaces rather than a shared file.

7.1 Ownership

Every owner holds a technical lane. Deliverables are distributed rather than assigned to one person: each owner writes their own README section and records a 60–90 second video segment covering their component, which any available member stitches together on the final day.

Owner	Lane	Scope	Critical path note
A	Retrieval Offline stage, Step 4	`product_text()`, FTS5 extension, embedding matrix, three streams, union, floor check.	Heaviest component; everything depends on it. Starts first and stubs immediately so nobody blocks.
B	State & routing Steps 1–3	Slot extraction, write/overwrite/delete, negation detection, canonical reconstruction, track rule.	Owns the `intent_override` path — 30 sessions, all `hard`, 15% of score. The principal differentiator.
C	Ranking Step 6	Ten features, logistic regression fit, pairwise comparison, flagged LLM rerank.	Develops offline against a fixed candidate list until A lands. Blocked on D for the feature matrix.
D	Simulator & training corpus §6.5, Step 7	User simulator (if required), telemetry logging, instrumented runs, feature matrix generation.	**On the critical path.** No weight can be fitted and no ablation run until D delivers the corpus.
E	Evaluation & integration Step 5, §6.3	Clarification policy, ablation harness, threshold tuning, module wiring, repository health.	Owns `main`. Good stubs reduce integration to review, leaving capacity for the clarification and ablation work.

WHY EVERY LANE IS TECHNICAL

An owner dedicated solely to documentation and video is idle for the first two days and overloaded on the third, while the technical owners lose the context needed to describe their own components accurately. Distributing the deliverables — each owner documenting and demonstrating what they built — produces better material and removes a single point of failure on the final day. The brief's requirement to record team member contributions is satisfied by issue assignment rather than by a dedicated scribe.

WHY D OWNS THE SIMULATOR RATHER THAN E

If the kit ships no user simulator (§6.5.1), building one is a substantial engineering task and a hard prerequisite for every fitted weight and every ablation row. Pairing it with telemetry keeps the corpus producer and the corpus consumer's upstream dependency in one place. Clarification moves to E, whose ablation harness is the mechanism that calibrates its thresholds.

7.2 Interface contracts, agreed before implementation

Half a day spent fixing these prevents the failure mode where five contributors build against divergent assumptions. All four are stubbed to return fixture data on day one, so the pipeline runs end to end before any component is real.

```
product_text(row) -> str # A — single shared definition update_slots(state, message) -> state # B pick_track(state) -> "buy"|"browse" # B retrieve(state, track) -> [candidate] # A rank(candidates, state) -> [asin] * 10 # C simulate_turn(session, history) -> str # D log_turn(session_id, turn, candidates) -> None # D pick_attribute(pool, state) -> str | None # E
```

THE SINGLE HIGHEST-RISK SHARED OBJECT

`product_text()` feeds the keyword index, the embedding matrix and the ranking blob. If one contributor alters its definition and only one index is rebuilt, the three views of the catalogue silently disagree — a defect that surfaces as degraded scores with no obvious cause. It lives in a shared utilities module and is changed only by agreement.

7.3 Schedule

Day	Milestone	Rationale
Day 1 AM	Interfaces agreed; all modules stubbed; baseline agent scored under the supplied evaluator.	A number on the board before any real work. Environment and evaluator problems surface here rather than later.
Day 1 PM	Parallel build begins. Embedding matrix encoded and persisted to `.npy`.	Encoding is the only step with an external model download; failure here must be discovered early.
Day 1 EOD	**Integration checkpoint.** End-to-end run with hardcoded ranking weights.	Late integration is the one failure that is not recoverable. Everything else can be descoped.
Day 2 AM	Simulator resolved (§6.5.1); instrumented run over 200 sessions; feature matrix produced.	Unblocks the ranker fit. Transcripts from `intent_override` and `boundary` sessions read at this point.
Day 2 PM	Ranker fitted; ablation table populated; thresholds tuned.	Tuning is only meaningful once the pipeline is stable.
Day 3	Each owner records their segment and writes their README section; assembly, Devpost, freeze.	Deliverables carry substantial weight. Distributing them keeps the final day parallel rather than serialised on one person.

7.4 Descoping order

If the schedule compresses, components are dropped in this order. Each is independently removable because it sits behind a flag or contributes a single feature.

Order	Component	Cost of removal
1	`rare_tag_match`	Negligible. Active in ~22% of sessions on a signal resting on 12–26 observations per tag.
2	LLM rerank	The regression already produces a complete ranking. Removal also improves Efficiency.
3	Answerability weighting	Falls back to pool-size threshold plus category entropy — most of the benefit at a fraction of the work.
4	Fitted ranker weights	Falls back to hand-set weights. Measurable loss, but the pipeline remains correct.
—	**Not descopable**	Hybrid retrieval, slot override, and the clarification decision. These constitute the pillars.

8. Detailed work breakdown

Each owner's deliverables, ordered by dependency, with the technology used at each step. Steps marked `BLOCKING` gate another contributor's work and take priority over that owner's later items.

8.0 Shared toolchain

Tool	Used for	Convention
GitHub	Source of truth; required as a public repository deliverable.	One branch per owner (`feat/retrieval`, `feat/state`, …), PR into `main`. `main` must always run end to end.
GitHub Issues / Projects	Task tracking against the steps below.	One issue per numbered step; label by owner. Provides the contribution record the README requires.
Git LFS or release assets	The 75MB embedding matrix and the 60MB catalogue.	Never commit large binaries to the repo directly; ship a `make data` download script instead.
Claude Code	Repository scaffolding, issue creation via `gh`, implementation of well-specified steps.	Given the design document as context, it scaffolds the module skeleton and opens the issue set in minutes. Suited to A3–A6, C2, E1–E4. Not to B5 or the clarification policy, where a plausible-looking wrong answer fails silently.
GitHub CLI (`gh`)	Issue and project-board creation from the §8 step tables.	`gh auth login` once per machine. Issues assigned by owner produce the contribution record the README requires, without a separate bookkeeping effort.
Python 3.11 + venv	Runtime.	Pinned `requirements.txt`; one command to set up. Version drift across five machines is a predictable time sink.
pytest	Regression tests, chiefly for state transitions.	Every override case that is fixed gains a test, so it cannot regress.
ruff + black	Lint and format.	Pre-commit hook. Removes formatting noise from diffs across five contributors.
Jupyter / marimo	Analysis: ablation tables, slice plots, feature distributions.	Notebooks for exploration only; anything the agent depends on is promoted to a module.

> **VERSION CONTROL DISCIPLINE**
>
> With five contributors over three days, the dominant risk is not individual defects but divergence — two people building against different assumptions and discovering it at integration. Short-lived branches, a continuously working `main`, and the fixture-stub approach in §7.2 are what keep that risk bounded.

8.1 Owner A — Indexes and retrieval

Deliverable: `indexes.py`, `retrieval.py`, `utils.py`. **Depends on:** nothing. **Blocks:** C and D.

#	Step	Technology	Definition of done
A1	**BLOCKING** Write `product_text()` in `utils.py` and publish the definition to the team.	Plain Python	All three indexes import it. No local copies exist anywhere in the repo.
A2	**BLOCKING** Stub `retrieve(state, track)` returning fixture ASINs.	Plain Python	C and D can develop against a stable signature within the first hour.
A3	Build the facts dictionary and per-category popularity lists.	Plain dicts; `log1p` precomputed	50,000 records; `dept` sourced from `categories[1]`, not `details`. Category lists pre-sorted by `rating_number`.
A4	Extend the FTS5 index; expose scores rather than a fixed `LIMIT top_k`.	SQLite FTS5, `unicode61`	Returns `(asin, score)` pairs. Sign corrected once at the boundary so higher is better throughout.
A5	Encode the catalogue; persist as `.npy`.	`sentence-transformers`, `all-MiniLM-L6-v2`, `numpy`	`(50000, 384)` float32, L2-normalised. Loads from disk in under one second on subsequent runs.
A6	Implement brute-force kNN search.	`numpy` matmul + `argpartition`	Top-k in roughly 5 ms. No ANN index, no external service.
A7	Implement the three streams with per-track quotas.	Owner's own modules	Keyword over-fetches 3× before filtering. Semantic and popularity never filter.
A8	Implement union, dedupe, and the floor check.	Plain Python	Pool is never empty. If filtering undershoots, the least-confident constraint is relaxed and retrieval repeats.
A9	Add category diversity on the browsing track.	MMR or per-category cap	No single category exceeds a set share of the browsing pool.

> **SEQUENCING NOTE FOR A**
>
> A5 requires downloading a model from Hugging Face. Attempt it on day one regardless of progress elsewhere — a blocked download discovered mid-build costs the semantic stream entirely. A1 and A2 must land within the first hour; everything else in the project waits on them.

8.2 Owner B — State and routing

Deliverable: `state.py`, `extract.py`. **Depends on:** A1. **Blocks:** A (needs slot shape), D.

#	Step	Technology	Definition of done
B1	**BLOCKING** Define the slot schema and publish it.	`dataclass` or typed dict	Fixed key set. A knows which slots may filter; C knows which contribute to `slot_coverage`.
B2	Build the attribute gazetteer from the catalogue itself.	Counter over `product_text()` output	Every extractable value provably exists in the data, which prevents hallucinated constraints.
B3	Implement rule-based extraction.	`re` + gazetteer lookup	Agent is fully functional with zero LLM calls. This is the fallback the whole system rests on.
B4	Implement merge policy: write, overwrite, delete.	Plain Python	Overwrite sets an override flag. Negation removes the key outright rather than adding a negative value.
B5	Implement the negation detector.	`re` patterns, refined from transcripts	Catches the constructions the simulator actually produces — determined by reading D8 transcripts, not guessed.
B6	Implement the scenario buffer with replace semantics.	Plain Python	A new scenario statement replaces the prior one. Verified by test.
B7	Implement `render()` and canonical reconstruction.	String join + MiniLM encode	Output string is logged verbatim every turn, making override behaviour directly auditable.
B8	Implement the routing rule.	Plain Python	Re-evaluated per turn. Validated against `scenario_type` on the public set, never read at inference.
B9	Optional: LLM extraction path behind a flag.	Constrained-JSON LLM call	Falls back to B3 on parse failure, timeout, or missing credentials.
B10	Write regression tests for every override	`pytest`	Each fixed case gains a test. Protects the 15% of

case. sessions that are `intent_override`.

WHY B OWNS THE HIGHEST-VALUE DEFECT SURFACE

All 30 `intent_override` sessions carry `difficulty_bucket: hard`, and they are the only sessions that do. If B5 fails to recognise a construction the simulator uses, the stale constraint persists and the target is lost for the remainder of the session — a failure that is invisible in aggregate metrics but concentrated in 15% of the score. Reading the transcripts (D8) is therefore a hard prerequisite for finalising B5.

8.3 Owner C — Ranking

Deliverable: `features.py`, `rank.py`, `fit.py`. **Depends on:** A2 to begin, D6 for the training corpus.

#	Step	Technology	Definition of done
C1	**BLOCKING** Define the candidate object shape.	`dataclass`	Carries ASIN plus each stream's contributed score. A populates it; C consumes it.
C2	Implement the ten feature functions.	Plain Python + `numpy`	Each independently unit-testable. `price_fit` returns a neutral value for nulls — never excludes.
C3	Implement scoring with hand-set weights.	Weighted sum	Constants at module top for single-line tuning. Produces a complete ranking on day one.
C4	Handle missing stream scores.	Default to zero	Popularity-stream candidates carry `bm25 = 0` and compete on other features, as intended.
C5	Fit the logistic regression on D6's matrix.	`scikit-learn`, `StandardScaler`, `class_weight='balanced'`	Scaler persisted alongside the model. Validated with `GroupKFold` grouped by `session_id`.
C6	Compare against a pairwise objective.	`lightgbm` `lambdarank`, grouped by session	Whichever wins on development MRR is retained. Time-permitting.
C7	Implement the LLM rerank behind a flag.	LLM API; defensive JSON parsing	Falls back to the regression ordering on malformed output. Token counts reported in `usage`.
C8	Report feature weights and correlations.	Jupyter, `pandas`	Near-zero weights identify inert features without requiring an ablation run.

8.4 Owner D — Simulator and training corpus

Deliverable: `simulate.py`, `telemetry.py`, `features.jsonl`. **Blocks:** C5 and all of E's ablation work.

#	Step	Technology	Definition of done
D1	**BLOCKING** Read the evaluator source; resolve the dialogue source (§6.5.1).	Code reading	Configuration A, B or C established. Also resolves turn counting, the `ask_attribute` vocabulary, and the Efficiency formula.
D2	Implement append-only telemetry.	JSONL	Logs state, canonical intent string, pool sizes, and **whether the target was in the pool**. Labels joined offline by `session_id`; ground truth never reaches `respond()`.
D3	Build the facet extractor over target records.	`re` + category-path parsing	Pulls department, category, colour, material and detail phrases from a target's catalogue record.
D4	**BLOCKING** Implement the user simulator with per-scenario release policies.	Plain Python; optional LLM paraphrase	Override sessions **always** emit a contradiction — falling back to category or department when the target carries no extractable colour. Emits singular nouns.
D5	Implement clarification answering.	Plain Python	Answers from the target record when it carries the asked attribute; returns "no preference" otherwise. This is what makes the answerability prior measurable.
D6	**BLOCKING** Run instrumented sessions; produce the feature matrix.	Own loop over `reset()` / `respond()`	~4,200 labelled rows delivered to C. Negatives sampled **from the candidate pool**, never at random from the catalogue.
D7	Report per-stream recall.	`pandas`	Which streams contained the target, and each stream's *unique* catches. Establishes the pool-recall ceiling that bounds Hit Rate@10.
D8	Circulate override and boundary transcripts.	Plain text export	Feeds B5's negation patterns and E's answerability priors. Both are otherwise guesswork.

> **WHY D1 PRECEDES EVERYTHING ELSE IN THIS LANE**
>
> Under configuration A the corpus is a logging change and D4–D5 are unnecessary. Under configuration C the simulator is a required component and this lane roughly doubles in size. The distinction is settled by a short source read, and it determines whether the schedule holds.

> **THE BIAS D MUST DOCUMENT**
>
> A self-built simulator draws its phrasing from the target's own record, making retrieval easier than reality and inflating internal Hit Rate. D reports this alongside any internally-generated score. Headline figures come only from the supplied evaluator; ablations remain valid because the bias applies equally across configurations.

8.5 Owner E — Evaluation and integration

Deliverable: `clarify.py`, `ablate.py`, `agent.py`, the results notebook. **Depends on:** D6 for ablation input.

#	Step	Technology	Definition of done
E1	**BLOCKING** Repository skeleton, pinned requirements, one-command setup.	Claude Code, GitHub, venv	Every contributor reproduces the environment identically. Scaffolded from this document.
E2	**BLOCKING** Wire stubs into a runnable `Agent`.	Kit's `reset()` / `respond()` contract	End-to-end run with fixture data before any component is real.
E3	Stand up the evaluator; record the baseline score.	Supplied evaluator	A number exists before development begins. Environment problems surface immediately.
E4	Implement the clarification policy.	Shannon entropy over pool attributes	Answerability priors initialised by judgement, then re-estimated from D8 transcripts. Per-session cap on clarifications.
E5	Build the ablation harness.	Config flags + evaluator	Feature ablations re-score logged candidates without re-running retrieval; stream and filter ablations trigger a full pass.
E6	Populate the ablation table; slice by `scenario_type`.	Jupyter, `pandas`	Nine configurations × four metrics, plus four slices. The core evidence in the writeup.
E7	Grid-search thresholds and stream quotas.	Coordinate ascent on dev	Locates the Hit Rate / MTTC frontier. Chosen values recorded with justification.
E8	Integrate modules; keep `main` green.	Git, PR review	`main` runs at all times. Integration completes on day one, not the final day.
E9	Data download script; verify from a clean clone.	`make` or shell; SHA256	Setup and reproduction confirmed on a machine that never built the project.

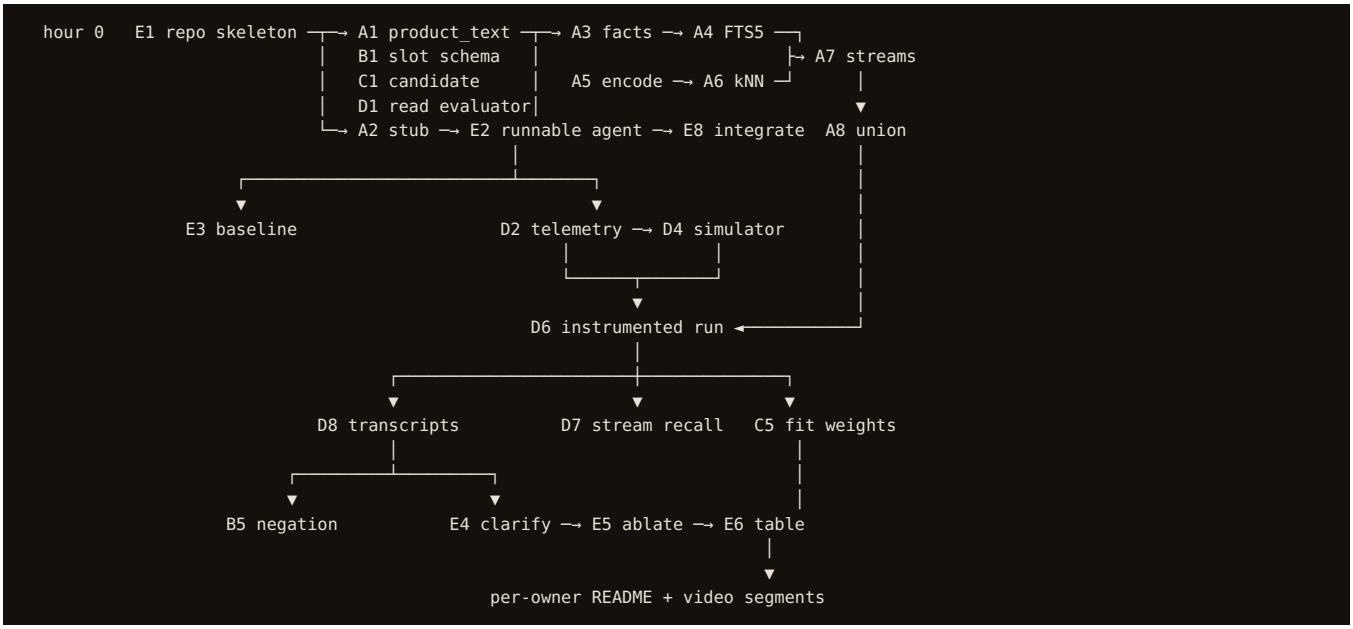
8.5.1 Distributed deliverables

No single owner is responsible for documentation or video. Each contributes material covering their own lane, and E assembles.

Artifact	Who produces it	Assembly
README sections	Each owner writes the section covering their modules, including its limitations.	E merges; the reflection required by the brief is drawn from the per-lane limitations.
Video segments	Each owner records 60–90 seconds demonstrating their component.	Any available member stitches. Required content: an `intent_override` session showing the canonical string losing the superseded constraint, and the ablation table.
Devpost description	Assembled from the merged README.	E, on the final day.
Contribution record	Generated from closed issues, assigned by owner.	Automatic, provided issues are used from day one.

> **WHAT THE DEMO MUST SHOW**
>
> A working search is unremarkable. Two things distinguish the submission: an `intent_override` session where the canonical intent string visibly loses the superseded constraint, and the ablation table demonstrating each component's measured contribution. The first proves the architecture does what it claims; the second proves it was verified rather than asserted.

8.6 Dependency graph



THE CRITICAL PATH

`E1 → A2 → E2 → D2 → D4 → D6 → C5 → E6`. Every fitted weight and every ablation row depends on the instrumented run, which depends on the simulator, which depends on a working end-to-end agent, which depends on stubs existing in the first hour. `D1` sits alongside as a gate: its outcome determines whether `D4` exists at all. Delay anywhere on this chain propagates directly to the evidence in the writeup. Work off the critical path — A9 diversity, C6 pairwise objective, B9 LLM extraction, E7 grid search — is deferred without consequence.

9. Known limitations

- **Slot extraction is the weakest link.** A hallucinated constraint on the buying track causes a hard filter to delete the target irrecoverably. Mitigated by gazetteer-constrained extraction and by falling back to browsing behaviour when extraction confidence is low.
- **Popularity may under-serve long-tail intent.** 4% of targets sit outside the top 20% by review count. Popularity is therefore never a filter, but the prior will still disadvantage genuinely obscure targets.
- **Boundary sessions test an undocumented capability.** Ten sessions, statistically indistinguishable from the rest on every catalogue dimension we examined. We handle them defensively — never return an empty list, never let a filter produce zero, keep popularity soft — and intend to read the transcripts from an instrumented run.
- **Answerability priors are hand-initialised.** They cannot be estimated statically because sessions contain no dialogue, so initial values are judgement calls refined from simulator logs.
- **Effective sample size is ~200, not 4,200.** Rows within a session share a query and are not independent. We regularise strongly and split by session.
- **Training corpus may be self-generated.** If the kit ships no user simulator, our own supplies the queries. Its phrasing derives from target attributes and is therefore easier to retrieve against than genuine user language, inflating internal Hit Rate. Headline figures come only from the supplied evaluator.
- `rating_style_fit` **is small.** 0.131 stars, about 16% of one catalogue IQR. Statistically significant at p=0.003 but of modest practical size; retained because it costs nothing and the regression can zero it.
- **Profile signal is thin and may not transfer.** `rare_tag_match` is active in ~22% of sessions and rests on 12–26 observations per tag. It is retained because the cost is negligible and the regression can zero its weight.
- **Cross-session personalization is out of reach.** Sessions are isolated single-user interactions, so adaptation is confined to within-session strategy adjustment — which is where the evidence actually is.